SYSTEM DESIGN DOCUMENT · V0.1 DRAFT

Vehicle Tracker Platform

Java backend · Multi-protocol GPS ingestion · Real-time WebSocket distribution · Web management console

Author: Serkan Yücel · Date: 6 November 2026 · Status: For review

1. Executive summary
2. Goals, scope & requirements
3. The 200+ protocol problem
4. High-level architecture
5. Component design
6. Data flow
7. Real-time WebSocket contract
8. REST API surface
9. Data model & storage
10. Management web application
11. Security
12. Scalability & capacity
13. Deployment & operations
14. Technology stack
15. Risks & open questions

1. Executive summary

Vehicles are fitted with third-party GPS tracker devices. Each device opens a TCP or UDP connection to our backend and periodically sends location reports in its vendor's protocol. The backend decodes every report into one normalised *Position* record, stores it, and pushes it in real time over WebSocket to every authenticated management client that is allowed to see that vehicle. The management application is a browser app: users sign in and watch their fleet move live on a map, with the map provider swappable.

> **Key decision:** we will not write 200+ protocol decoders ourselves. The open-source **Traccar** server (Java, Apache 2.0) already ships decoders for 200+ protocols and 2,000+ device models. We embed it as a stateless **Ingestion Gateway** whose only job is *bytes in → normalised Position out*, published to Kafka. Everything downstream (business logic, storage, real-time push, auth, UI) is our own code and does not depend on Traccar internals.

2. Goals, scope & requirements

2.1 Functional requirements

ID	Requirement	Phase
F1	Accept connections from tracker devices over TCP and UDP using any of the 200+ supported GPS protocols.	MVP
F2	Identify each device (IMEI / unique ID) and map it to a registered vehicle; reject or quarantine unknown devices.	MVP
F3	Normalise every report to one Position schema (time, lat, lon, speed, heading, altitude, satellites, ignition, attributes).	MVP
F4	Publish positions to connected management clients via WebSocket within ~1 s of receipt.	MVP
F5	Users authenticate before any access; they see only vehicles in their organisation / group.	MVP
F6	Display live vehicles on a map; map provider is pluggable (OpenStreetMap, Google, Mapbox, Bing, HERE ...).	MVP
F7	Manage vehicles, devices, users, groups via the web app.	MVP
F8	Persist position history; replay a vehicle's route for a time range.	v1
F9	Events & alerts: online/offline, ignition, overspeed, geofence enter/exit, SOS.	v1
F10	Send commands to devices (e.g. engine stop, set interval) where the protocol supports it.	v2
F11	Reports (trips, stops, mileage), public REST API & webhooks for integrators.	v2

2.2 Non-functional requirements

Quality	Target (initial)
Scale	Design for 100,000 concurrent devices; launch sized for 10,000. Horizontal scaling for every tier.
Latency	Device packet → browser marker update: p95 < 1 s (excluding the device's own reporting interval).
Availability	99.9 % for ingestion and live view. No single point of failure.
Durability	No acknowledged position is lost. Devices are ACKed only after the event is written to Kafka.
Security	OIDC authentication, tenant isolation, TLS for all client traffic, audit log of admin actions.
Retention	Raw positions 12 months (configurable per tenant); downsampled history longer.
Extensibility	New protocol support by upgrading the gateway dependency, without touching other services.

2.3 Out of scope (for now)

- Native mobile apps (the web app will be responsive; a mobile app can come later on the same APIs).
- Building our own hardware or firmware.
- Video telematics / dashcam streams (protocol family is different; revisit in a later phase).

3. The 200+ protocol problem

Tracker vendors each define their own wire format: binary (Teltonika Codec 8/8E, Concox/GT06, Queclink), ASCII (TK103, Meitrack, NMEA-like), HTTP-based (OsmAnd), and many variants of each. Every protocol has its own framing, login handshake, checksum, ACK rules, and set of extra attributes (fuel, CAN bus, temperature, RFID driver ID). Writing and maintaining 200 of these is years of work and requires physical test devices.

3.1 Options considered

Option	Description	Pros	Cons
A. Write our own decoders	Netty server + hand-written decoder per protocol.	Full control, no external code.	Years to reach 200 protocols; huge test burden. Not realistic.
B. Run Traccar as the whole product	Deploy stock Traccar, customise its web UI.	Fastest start; feature-rich.	Monolith with its own data model, auth, UI; hard to scale horizontally or differentiate; we inherit its architecture.
C. Traccar as Ingestion Gateway Recommended	Use Traccar only for protocol decoding. Forward each decoded position to Kafka. Our own services do the rest.	200+ protocols on day one; clean separation; our services scale independently; upgrades bring new protocols.	Must manage the boundary carefully (version pinning, mapping its Position model to ours, command downlink path).

3.2 How the gateway works (option C)

- Traccar runs one Netty pipeline per protocol, each listening on its own port (e.g. 5023 for GT06, 5027 for Teltonika). Devices are configured with our host and the port of their protocol.
- When the protocol is unknown, a device can still be identified using Traccar's tooling against captured packets; ops then assign it to the right port.
- The gateway runs *stateless with respect to business data*: device identity is checked against a cache fed from our Device Registry; it does not own users, vehicles or history.
- Decoded positions are published to Kafka. Two integration paths, decided in a spike (see §15):
    1. **Configuration-only:** use Traccar's built-in position/event forwarding (it supports several forwarding targets, including message brokers, in recent versions — to be verified against the pinned version).
    2. **Thin fork / extension:** depend on Traccar's protocol module and add our own Netty handler at the end of the pipeline that maps `Position` → our Avro/Protobuf schema and produces to Kafka synchronously before the device is ACKed. This gives us the durability guarantee in §2.2.
- Licence: Apache 2.0 allows commercial use and modification; we must keep the licence and NOTICE files and mark modified files.

4. High-level architecture

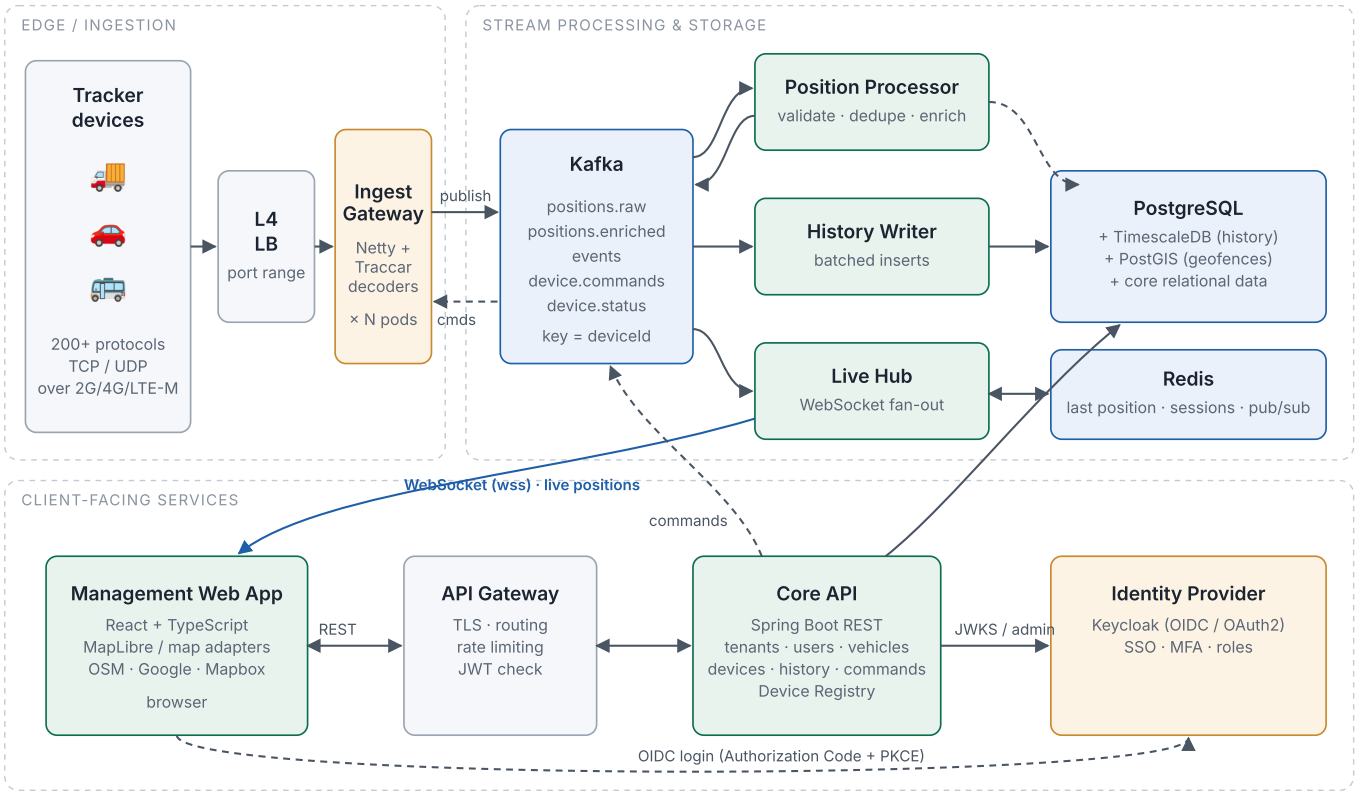


Figure 1 — Logical architecture. Green = services we build; amber = adopted open-source components; blue = data infrastructure. Dashed lines = control/secondary paths.

The design is event-driven around Kafka. The ingestion tier is deliberately thin and replaceable; Kafka decouples the bursty, connection-heavy device side from the client side. Every service is stateless and horizontally scalable; state lives in Kafka, PostgreSQL and Redis.

5. Component design

5.1 Ingestion Gateway (Traccar-based)

- **Responsibility:** hold long-lived device connections, run protocol decoders, send protocol-level ACKs, publish normalised positions, deliver downlink commands to connected devices.
- **Device identity:** the unique ID from the login/first packet is looked up in a local cache (Caffeine) loaded from the Device Registry (Core API) and kept in sync via the `device.registry` Kafka topic. Unknown IDs are published to `device.unknown` for ops review instead of being silently dropped.
- **Session ownership:** on connect, the gateway writes `deviceId → podId` to Redis (TTL refreshed on traffic) so command delivery can be routed to the right pod.
- **Durability:** Kafka producer with `acks=all`, idempotence on; device ACK is sent only after the produce completes. If Kafka is unreachable, the gateway stops ACKing and devices buffer locally (most trackers store hundreds to thousands of points).
- **Scaling:** N identical pods behind a Layer-4 load balancer. TCP connections are sticky by nature; UDP protocols need source-IP affinity.

5.2 Position Processor

- Kafka Streams (or Spring Kafka consumers) reading `positions.raw`, partitioned by `deviceId` so each device's points are processed in order.
- **Validate:** drop invalid fixes (lat/lon 0,0, no GPS fix, future timestamps), flag outliers (impossible jumps by speed).
- **Dedupe:** devices resend buffered points after reconnecting; dedupe by `(deviceId, fixTime)`.
- **Enrich:** attach `vehicleId`, `tenantId`, `groupIds`; compute derived fields (odometer delta, moving/stopped state).
- **Events (v1):** online/offline, ignition on/off, overspeed, geofence enter/exit (PostGIS geometries cached in memory), alarms from device attributes. Output to `events`.
- Writes `positions.enriched`; late/historical points (buffered offline data) are marked `historical=true` so the live map does not jump back in time.

5.3 History Writer

- Consumes `positions.enriched` and `events`, writes in batches (e.g. 500 rows / 200 ms) to TimescaleDB using COPY or batched inserts.
- Idempotent upserts on `(device_id, fix_time)` so reprocessing is safe.

5.4 Live Hub (WebSocket service)

- Spring Boot with Spring WebSocket (STOMP over WebSocket, or raw WebSocket with a small JSON protocol — see §7). Reactive (WebFlux/Netty) for high connection counts.
- Each pod consumes `positions.enriched` and `events` with its *own* consumer group (broadcast) and forwards only to its locally connected sessions whose authorisation scope includes the vehicle. Alternative at larger scale: a router publishes to Redis channels per tenant and pods subscribe only to tenants they hold sessions for.
- **Snapshot on connect:** on subscribe, the client receives the last known position of every visible vehicle from Redis, then incremental updates.
- **Back-pressure:** per-session bounded queue; if a slow client falls behind, older updates for the same vehicle are coalesced (only the latest matters for a live map).
- **Optional viewport filtering:** clients may send their map bounds; the hub only sends vehicles inside the box (+ margin). Important for tenants with tens of thousands of vehicles.

5.5 Core API

- Spring Boot 4 REST service (Spring MVC, Spring Security as OAuth2 Resource Server).
- Owns tenants, users (profile; credentials live in the IdP), roles, groups, vehicles, devices (Device Registry), geofences, history queries, command requests, audit log.
- Publishes registry changes to `device.registry` for gateway caches; publishes command requests to `device.commands`.

5.6 Identity Provider

- Keycloak (or a managed OIDC provider: Auth0, Entra ID, Cognito). Supports SSO for enterprise customers, MFA, password policy.
- Access token (JWT, ~5 min) carries `sub`, `tenant_id`, roles. Fine-grained group access is resolved by Core API / Live Hub and cached.

6. Data flow

6.1 Uplink: device report → live map

#	Step	Component
1	Device connects to `track.example.com:5027` and sends login + location packet.	Device → L4 LB
2	Protocol decoder frames and decodes packet; unique ID resolved against the registry cache.	Ingestion Gateway
3	Normalised Position produced to `positions.raw` (key = deviceId, acks=all).	Gateway → Kafka
4	Protocol ACK returned to device.	Gateway → Device
5	Validate, dedupe, enrich with vehicle/tenant; derive events; produce to `positions.enriched`.	Position Processor
6a	Batch-written to TimescaleDB.	History Writer
6b	Last-known position updated in Redis; pushed to authorised WebSocket sessions.	Live Hub
7	Browser moves marker (with smooth interpolation), updates side panel.	Web App

6.2 Downlink: user command → device (v2)

1. User clicks “Request position” / “Immobilise” → `POST /api/v1/devices/{id}/commands` (permission check, audit log, optional 2-step confirmation for dangerous commands).
2. Core API writes the command (status *PENDING*) and produces to `device.commands`.
3. The gateway pod that owns the session (looked up in Redis, or all pods filter by deviceId) encodes it in the device's protocol and sends it. If the device is offline, the command is queued until it reconnects (or expires).
4. Device response → `device.status` → Core API updates status → Live Hub notifies the user.

7. Real-time WebSocket contract

Endpoint: `wss://app.example.com/ws/live`. The access token is sent in the first message (not in the URL, to keep it out of logs). The server closes the socket when the token expires unless a refreshed token is sent.

```
// client → server
{ "type": "auth", "token": "<JWT>" }
{ "type": "subscribe", "scope": { "groups": ["g-12"], "vehicles": [] },
  "bounds": { "n": 51.70, "s": 51.28, "e": 0.33, "w": -0.51 } }   // bounds optional
{ "type": "bounds", "n": 51.6, "s": 51.4, "e": 0.1, "w": -0.3 }
{ "type": "ping" }

// server → client
{ "type": "snapshot", "positions": [ { ...position... }, ... ] }
{ "type": "position",
  "vehicleId": "v-8812", "deviceId": "d-3391",
  "fixTime": "2026-11-06T18:27:04Z", "serverTime": "2026-11-06T18:27:05.112Z",
  "lat": 51.50735, "lon": -0.12776, "speedKph": 42.5, "course": 118,
  "altitude": 21, "satellites": 11, "valid": true,
  "ignition": true, "motion": true,
  "attributes": { "fuelPct": 63, "batteryV": 12.6, "odometerKm": 48211.4 } }
{ "type": "event", "vehicleId": "v-8812", "kind": "GEOFENCE_EXIT",
  "time": "2026-11-06T18:27:04Z", "data": { "geofenceId": "gf-7" } }
{ "type": "status", "vehicleId": "v-8812", "online": false, "lastSeen": "..." }
{ "type": "pong" }
```

Positions are compact JSON initially; if bandwidth matters at scale, switch to a binary encoding (e.g. Protobuf / MessagePack) negotiated via WebSocket sub-protocol without changing message semantics.

8. REST API surface (Core API, v1)

Resource	Endpoints
Session / me	`GET /api/v1/me` — profile, tenant, roles, preferences (incl. preferred map provider)
Vehicles	`GET/POST /vehicles`, `GET/PUT/DELETE /vehicles/{id}`
Devices	`GET/POST /devices`, `PUT /devices/{id}`, `POST /devices/{id}/assign` (to vehicle), `GET /devices/unknown`
Positions	`GET /vehicles/{id}/positions?from=&to=&simplify=` (history, paged), `GET /positions/latest?group=`
Groups & users	`/groups`, `/users`, `/users/{id}/roles`
Geofences (v1)	`/geofences` (GeoJSON in/out)
Events (v1)	`GET /events?vehicle=&type=&from=&to=`
Commands (v2)	`POST /devices/{id}/commands`, `GET /devices/{id}/commands`
Protocols	`GET /protocols` — supported protocols, their ports, models (for device onboarding UI)

Documented with OpenAPI 3 (springdoc); the web app's TypeScript client is generated from it.

9. Data model & storage

9.1 Core entities (PostgreSQL)

```
tenant          (id, name, plan, retention_days, created_at)
app_user        (id, tenant_id, idp_subject, email, name, locale, map_provider, created_at)
user_group      (id, tenant_id, name)
user_group_member (group_id, user_id, role)          -- role: VIEWER | DISPATCHER | ADMIN
vehicle         (id, tenant_id, name, plate, vin, type, icon, group_id, created_at)
device          (id, tenant_id, unique_id /*IMEI*/, protocol, model, phone, sim_iccid,
                 status, last_seen_at, created_at)    -- UNIQUE(unique_id)
device_assignment (device_id, vehicle_id, from_ts, to_ts)   -- device swaps keep history
geofence        (id, tenant_id, name, area geography, created_at)
command         (id, device_id, type, payload jsonb, status, requested_by, created_at, updated_at)
audit_log       (id, tenant_id, user_id, action, entity, entity_id, data jsonb, at)
```

9.2 Time-series (TimescaleDB hypertable)

```
position (
  device_id   uuid        not null,
  vehicle_id  uuid,
  tenant_id   uuid        not null,
  fix_time    timestamptz not null,
  server_time timestamptz not null,
  lat double precision, lon double precision,
  speed_kph real, course smallint, altitude real, satellites smallint,
  valid boolean, ignition boolean,
  attributes  jsonb,
  primary key (device_id, fix_time)
)  -- hypertable on fix_time, 1-day chunks; compression after 7 days;
   -- retention policy per tenant; continuous aggregate for 1-minute downsampling.
```

Volume estimate: 10,000 vehicles × one point every 10 s while moving (~10 h/day) ≈ 36 M rows/day. At ~150 bytes/row uncompressed ≈ 5.4 GB/day; TimescaleDB compression typically cuts this by an order of magnitude. At 100k vehicles, consider a dedicated time-series cluster or ClickHouse for history.

9.3 Redis keys

Key	Value	Purpose
`pos:last:{vehicleId}`	Position JSON	Snapshot on connect; latest-positions API
`tenant:{id}:vehicles`	GEO set	Fast viewport queries (`GEOSEARCH`)
`session:{deviceId}`	gateway pod id (TTL)	Command routing
`authz:{userId}`	visible vehicle/group ids (TTL)	Live Hub authorisation cache

10. Management web application

10.1 Stack

- React + TypeScript, Vite build, served as static assets from CDN / object storage.
- Auth: OIDC Authorization Code + PKCE (e.g. `oidc-client-ts`); tokens in memory, silent refresh.
- State: TanStack Query for REST data; a dedicated live store (Zustand) fed by the WebSocket client.
- UI components: a mature component library (e.g. MUI or Mantine) for admin screens.

10.2 “Any map” — map abstraction

The UI talks to a small `MapAdapter` interface, not to a vendor SDK directly:

```
interface MapAdapter {
  mount(el: HTMLElement, opts: { center: LatLng; zoom: number }): void;
  upsertMarker(id: string, p: LatLng, style: MarkerStyle): void;
  removeMarker(id: string): void;
  drawTrack(id: string, points: LatLng[], style: LineStyle): void;
  fitBounds(b: Bounds): void;
  onBoundsChange(cb: (b: Bounds) => void): void;
  destroy(): void;
}
```

- **Default adapter: MapLibre GL JS** (open source, WebGL, handles thousands of markers via GeoJSON sources/clustering). With raster/vector tile sources it covers OpenStreetMap, MapTiler, Mapbox styles, Esri, HERE tiles, etc.
- Additional adapters for providers that require their own SDK: **Google Maps JS API**, optionally Leaflet for lightweight/legacy needs.
- Provider choice: tenant default + user preference; API keys per tenant are stored server-side and delivered via `/me`.
- Marker animation: interpolate between consecutive positions over the reporting interval so movement looks smooth; rotate icon by course.

10.3 Screens (MVP → v1)

Screen	Contents
Login	Redirect to IdP; tenant branding.
Live map	Full-screen map, vehicle list with search/filter (group, status, moving/stopped/offline), click vehicle → details card (speed, last update, address, driver, attributes), follow mode, clustering.
Vehicle history (v1)	Date-range picker, route polyline, playback slider, stops and events on the timeline.
Vehicles / Devices	CRUD, device onboarding wizard (choose model → shows server host/port to configure), assignment to vehicle, unknown-device inbox.
Users & groups	Invite users, assign roles and vehicle groups.
Geofences & alerts (v1)	Draw polygons/circles; alert rules; notification channels.

11. Security

Area	Measures
User access	OIDC; short-lived JWT; refresh via IdP; MFA for admins; RBAC (Viewer, Dispatcher, Admin, Super-admin).
Tenant isolation	Every row carries `tenant_id`; enforced in the repository layer and optionally PostgreSQL Row-Level Security; Live Hub filters on server side — never trusts client scope.
Device channel	Most GPS protocols are unauthenticated plaintext. Mitigations: only registered unique IDs accepted; anomaly detection (same IMEI from two IPs, impossible jumps); TLS ports for devices that support it; private APN / VPN with mobile operator for high-security customers.
Transport	TLS 1.2+ for all web/API/WebSocket traffic; WebSocket origin check; token in first message, not URL.
Commands	Dangerous commands (immobiliser) need ADMIN role, confirmation, and are fully audited.
Abuse	Per-IP connection limits at the gateway; API rate limits; WAF in front of web endpoints.
Privacy	Location data is personal data under UK/EU GDPR when linked to drivers: retention limits, data export/erasure per tenant, optional “private mode” for off-duty hours.

12. Scalability & capacity

	Launch (10k devices)	Target (100k devices)
Ingest rate (avg / peak)	~1k msg/s / 5k msg/s	~10k msg/s / 50k msg/s
Gateway pods	2–3 (≈ 10k conns each is comfortable for Netty)	10–15
Kafka	3 brokers, 24 partitions per position topic	3–6 brokers, 96+ partitions
Processor / Writer	2 + 2 instances	scale to partition count
Live Hub	2 pods (thousands of sessions each)	N pods; tenant-sharded routing via Redis
Database	1 primary + 1 replica	Primary + replicas; history possibly separate cluster

Ordering is guaranteed per device by Kafka partitioning on `deviceId`. All consumers are idempotent so at-least-once delivery is safe.

13. Deployment & operations

- **Packaging:** one Docker image per service; Kubernetes (EKS/GKE/AKS or self-managed) with Helm charts. Local dev via Docker Compose (Kafka, Postgres/Timescale, Redis, Keycloak).
- **Gateway networking:** needs hundreds of ports open. Use a cloud Network Load Balancer with a port range (or a dedicated node pool with `hostNetwork` + external L4 balancer such as HAProxy). Only enable ports for protocols actually in use to reduce attack surface.

- **Managed options:** managed Kafka (MSK, Confluent, Aiven), managed Postgres with TimescaleDB (Timescale Cloud) and managed Redis reduce ops load.
- **Observability:** Micrometer → Prometheus/Grafana; OpenTelemetry tracing across services (trace id stamped at the gateway); structured JSON logs → Loki/ELK. Key metrics: connected devices per protocol, decode errors per protocol, ingest lag, Kafka consumer lag, end-to-end latency, WebSocket sessions.
- **Raw packet capture:** optional per-device hex logging (time-limited) for troubleshooting new device models.
- **CI/CD:** GitHub Actions → build, test, container scan → deploy to staging → production with canary for gateway (device connections drain gracefully).
- **Testing:** device simulators replaying recorded packets per protocol; load tests (Gatling/k6) for 100k simulated connections and WebSocket clients.

14. Technology stack

Layer	Choice	Notes
Language / runtime	Java 21 LTS	Virtual threads for blocking I/O in REST services.
Framework	Spring Boot 4.x	Spring Security, Spring Kafka, Spring WebSocket/WebFlux, springdoc.
Protocol decoding	Traccar server (Apache 2.0), Netty	Pinned version; upgraded deliberately with regression tests.
Messaging	Apache Kafka	Schemas in Avro/Protobuf with a schema registry.
Databases	PostgreSQL + TimescaleDB + PostGIS	One engine for relational, time-series and geo initially.
Cache / pub-sub	Redis	Last positions, GEO index, session routing.
Identity	Keycloak (OIDC)	Or managed IdP.
Frontend	React, TypeScript, Vite, MapLibre GL JS	Pluggable map adapters (Google, Leaflet ...).
Infra	Docker, Kubernetes, Helm, Terraform	GitHub Actions CI/CD.
Observability	Prometheus, Grafana, OpenTelemetry, Loki	

15. Risks & open questions

Risk / question	Impact	Mitigation / next step
Integrating Traccar as a library vs. using built-in forwarding	Durability and upgrade effort	Spike in Phase 0: prototype both, measure, decide. Keep our mapping code in a separate module.
Upstream Traccar changes break our integration	Gateway upgrades	Pin versions; protocol regression suite with recorded packets; upgrade quarterly.
“Supports 200+ protocols” ≠ every device variant works	Customer onboarding	Certified device list; test with real hardware for top 10 models customers use.
Many open ports on the gateway	Security, cloud LB limits	Enable only needed ports; check cloud LB listener limits early.
UDP protocols behind load balancers	Responses to wrong source	Source-IP affinity; test per UDP protocol.
Live view for very large tenants	Browser performance	Viewport filtering, clustering, WebGL rendering.
Which device models will customers use first?	Prioritises testing	Open question for product.
Cloud vs. on-premise deployment for customers?	Packaging, licensing	Open question — affects multi-tenancy vs. single-tenant installs.
Commercial map provider costs (Google etc.)	Running cost	Default to OSM-based tiles; premium providers per tenant with their own keys.

Next document: delivery roadmap (phases, milestones, team, estimates). References: Traccar project — github.com/traccar/traccar.